

IOWA STATE UNIVERSITY

Aircraft Data Visualization

Review of Version Two / HCI584 / Andy Walker

Developers Adendum:

This project includes 3 different versions of the application, one command line, one using flask and focused on Streamlit. The Streamlit app is the main.py application. However the other versions are included to ease inclusion in other, larger projects.

The command line application can be found in the ./CLI folder and can be invoked with:

```
python .\command_line_app.py
```

Figure 1 gives example output, tabulated for easy reading.

```
PS C:\Users\andyw\OneDrive\Documents\Documents to Assign\! Class\HCI584\Andywalker_project\CLI> python .\command_line_app.py
```

Degrees (Az)	Degrees (El)	Slant Range (km)	2D Range (km)	Masked?	Received Time (UTC)
252.495	0.224	134.29	134.28	False	21:18:21.851
277.075	2.194	99.34	99.23	False	21:18:21.857
304.973	0.023	112.2	112.2	False	21:18:21.864
258.939	8.296	87.96	86.95	False	21:18:21.871
311.999	4.784	113.56	113.08	False	21:18:21.879
279.74	2.637	65.09	65.01	False	21:18:21.884
214.655	8.02	90.97	89.99	False	21:18:21.890
235.584	9.322	62.42	61.55	False	21:18:21.895
333.717	5.564	112.86	112.23	False	21:18:21.902
331.118	1.043	92.09	92.06	False	21:18:21.909
199.033	5.474	125.68	124.99	False	21:18:21.916
202.124	1.42	101.48	101.43	False	21:18:21.924
313.016	13.898	40.7	39.47	False	21:18:21.929
243.929	3.571	29.47	29.41	False	21:18:21.932

Figure 1: Example Output from CLI Version

The output uses Lat/Lon/Altitude to each aircraft to produce an Azimuth and Elevation pointing angle from the sensor to the aircraft. A Slant Range to the aircraft is also calculated to give an approximate location in 3-space of the aircraft to guide the sensor. The 'masked' indicator gives a sense of whether the aircraft should be visible to the sensor or not due to terrain masked, and the time stamp returns the time of the measurement.

The flask applications builds on the command line script, plotting the data of the aircraft with hotspot popups on a moving map. Invoking flask should yield output as shown in Figure 2.

```

PS C:\Users\andyw\OneDrive\Documents\Documents to Assign\! Class\HCI584\AndyWalker_project\Flask> python .\flask_app.py
* Serving Flask app 'flask_app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [02/Aug/2026 16:05:47] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:05:48] "GET /defaults HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:05:48] "GET /map HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:05:49] "GET /map_ts HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:05:49] "GET /map?ts=1785185991698 HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:05:52] "POST /update HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:05:52] "GET /map?ts=1785704752791 HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:05:53] "GET /map_ts HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:05:53] "GET /map?ts=1785704752790 HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:05:54] "POST /update HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:05:54] "GET /map?ts=1785704754726 HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:05:58] "GET /map_ts HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:05:58] "GET /map?ts=1785704754999 HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:06:03] "GET /map_ts HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:06:08] "GET /map_ts HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:06:13] "GET /map_ts HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:06:18] "GET /map_ts HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:06:23] "GET /map_ts HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:06:28] "GET /map_ts HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:06:33] "GET /map_ts HTTP/1.1" 200 -
127.0.0.1 - - [02/Aug/2026 16:06:38] "GET /map_ts HTTP/1.1" 200 -
PS C:\Users\andyw\OneDrive\Documents\Documents to Assign\! Class\HCI584\AndyWalker_project\Flask>

```

Figure 2: Expected Flask Server Invocation and Output

Figure 3 shows the hosted web page from the Flask server. As can be seen in the figure, no controls are available on the Flask served page, instead simply showing the location of the aircraft with their respective popups.

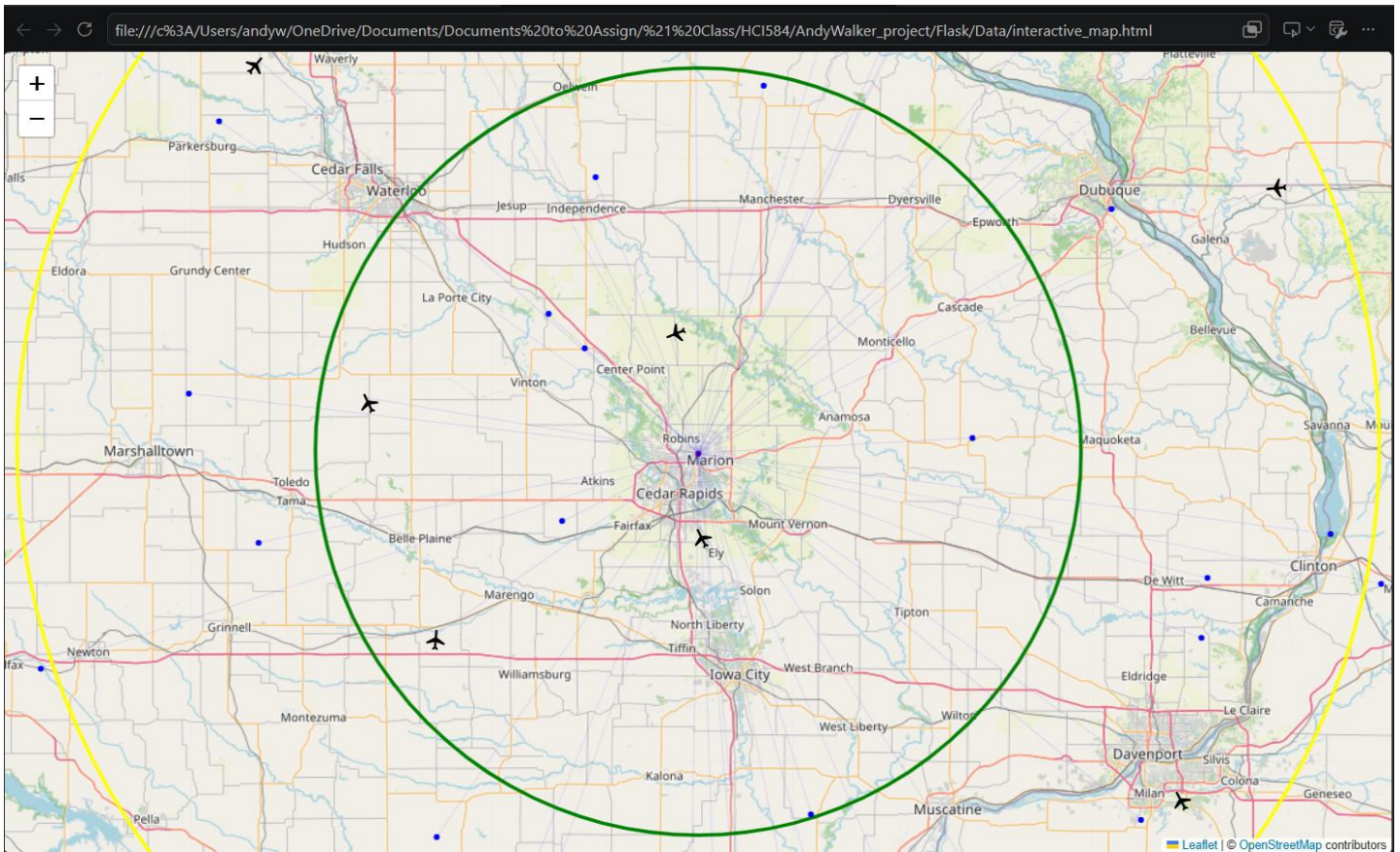


Figure 3: Flask Application User Interface

Command Line Functionality

Developers interested in using this work can directly utilize the functions available in the library by including it in their project:

```
``import MyModules``
```

Functions Implemented:

def calculate_slant_range(radar, plane): Calculates the azimuth and elevation angles from the sensor location to the aircraft as well as a slant range. The slant range is the range that the sensor would need to detect and track over.

def read_gps_coordinates(serial_port='COM5', baud_rate=4800, timeout=1, max_attempts=50): Reads the location of the sensor from a GPS receiver on COM5 (default). This output will be refined with information from the Iowa Real Time Kinematic Network in future work.

def calculate_radar_range(pt_watts=250, gain_db=26, num_pulses=1000, freq_hz=2.45e9, rcs_sqm=1.0, s_min_watts=1e-13, loss_db=0): Calculate the effective range of the sensor given 1000 coherent pulses. This is the equation used to generate the range 'stop-light' rings shown on the map.

def call_api(latitude, longitude, altitude, limit_range="75", units="M"): Collects the data from the ADS-B API.

def plot_map (latitude, longitude, range_10, range_20, range_30, my_map): Initializes 'my_map' and plots the range rings that show what aircraft are within detection range of the sensor.

def plot_plane (coord1, coord2, my_map, description, col = "blue", heading = 0): Plots the locations of the planes. One of the next steps is to add unobtrusive persistence to this plotting.

def get_masking(coord1, coord2, num_segments): Calculates the elevation at each point along a line projected on the ground between the sensor and the aircraft. If any of the elevations are higher than the elevation of the line of sight between the sensor and the aircraft, the line of sight is marked 'masked' and the aircraft is plotted in red to show that it is probably not visible to the sensor.

def filter_list(r): Consumes the JSON file 'r' and filters out the information from the ADS-B return that is used in the subsequent processing. Altitude, latitude, and longitude are copied from input to output. Heading is available but not used; instead the tracking algorithm should use a 'plausibility circle' to determine which aircraft past state corresponds to which aircraft current state.

def set_location(Port="COM5"): Has become the single function for defining the location of the center node. Accommodates GPS coordinates or variable defaults. Applies to AC_Range.py, app.py, and streamlit_app.py

def rotate_icon(angle): Rotates a PNG icon to an angle specified by the caller and saves (overwrites) the newly rotated image for use as an icon on a map.

def plot_vector(coord1, coord2, my_map, col = "blue"): Takes as input the location of an aircraft along with it's last location to generate a 'trail of ants' vector that points in the direction of travel. WIP.

def extract_matching_flights(file1_path, file2_path): Finds matching flights between two files and outputs a new file containing pairs of (lat, lon) coordinates for those flights.

Data/print_data.py: Formats the data.txt file for easy inspection

Data/print_output.py: Formats the output.txt file for easy inspection

Data Files Generated:

Ac_range.log: The formatted table of Azimuth and Elevation angles and slant ranges from the sensor location to each aircraft, including terrain masking bit and time of measurement in UTC.

Data.txt: A JSON file (a dictionary of lists of dictionaries) of data returned from the ADSB API call containing all the data of the aircraft within range of the sensor

Interactive_map.html: An HTML file that, once rendered, shows the locations of each aircraft, the range rings, and the popup content of each aircraft.

Output.txt: A filtered list of aircraft data including flight number, altitude according to GPS, the track the aircraft is flying, the latitude and longitude of each aircraft.

Output.old: The list of aircraft data from the prior API call. This data is preserved from call to call so the track can be verified. This method matches the track solution the sensor would generate, since no self-reported data is available to the sensor in realistic scenarios.

Some Anticipated Errors:

A range of errors will be returned if the data files are not available in the /Data file due to incomplete error checking for file presence and content. Therefore, once all packages are installed, any remaining errors should first be mitigated by copying old data files to the /Data directory.

Ongoing Work:

As of this writing, there are 2 outstanding tasks that should be reengaged:

- 1) The proper rotation of the aircraft to match the true and reported headings
- 2) The 'trail of ants' moving vector that illustrates a simple linear extrapolation of the current heading

Both of these have been working in previous development versions and only need debug to incorporate back into the main code line.

Next steps are described in other documents in the /Doc folder and focus on integration of the calculations and visualizations into a larger control UI for a real multi-static sensor.